Parallel And Distributed Computing

Assignment–2

Submitted to:

Dr. Saif Nalband

Submitted By:

Nitin Gupta
Roll no. : 102303525

Question 1: Molecular Dynamics - Force Calculation

Problem: Implement parallel computation of Lennard-Jones potential forces in a molecular dynamics simulation. Given N particles in 3D space, calculate the total potential energy and forces acting on each particle.

Tasks:

1. Parallelize the nested loops using OpenMP
2. Handle race conditions in force accumulation
3. Implement reduction for total energy
4. Optimize load balancing
5. Add performance measurement

Sol.

The Lennard–Jones (LJ) potential between two particles i and j is:

$$V(r) = 4\epsilon \left[\left(\frac{r}{\sigma}\right)^{12} - \left(\frac{r}{\sigma}\right)^{6} \right]$$

- r = distance between particles i and j
- The force is derived from the gradient of this potential
- Force on particle i due to j is equal and opposite to force on j due to i

Observation

The program output shows that execution time decreases as the number of threads increases, confirming that the computation benefits from parallel execution. The speedup improves significantly when moving from one thread to multiple threads and reaches a maximum value of approximately 3.65 at 16 threads.

Threads	Time (s)	Speedup
1	0.011799	1.00
2	0.007091	1.66
3	0.005862	2.01
4	0.005765	2.05
5	0.007111	1.66
6	0.005713	2.07
7	0.005564	2.12
8	0.004488	2.63
9	0.004143	2.85
10	0.004045	2.92
11	0.006806	1.73
12	0.010686	1.10
13	0.008157	1.45
14	0.003917	3.01
15	0.003327	3.55
16	0.003232	3.65

Figure 1: Execution Time and Speedup vs Threads

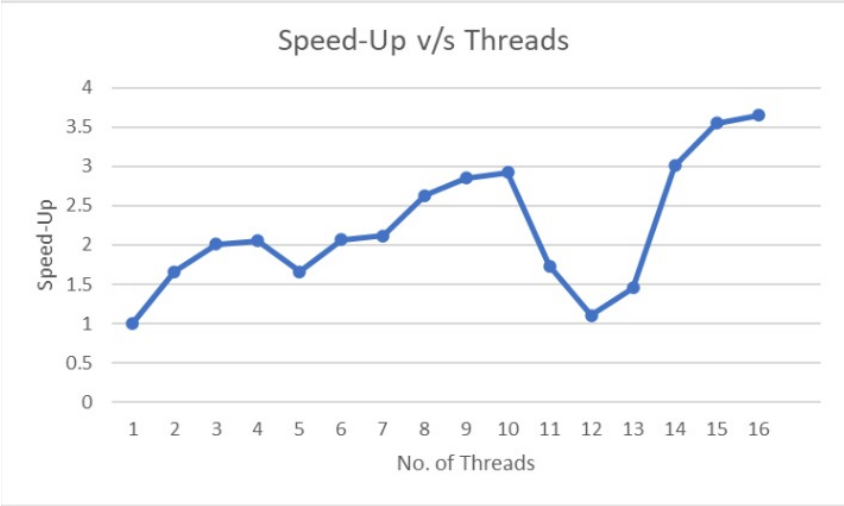

Figure 2: Speed-Up vs Threads

The speedup does not increase linearly and exhibits fluctuations at certain thread counts. This behavior can be explained by the following factors:

- The Lennard-Jones force calculation is memory-bound, meaning that performance is limited by memory access rather than computation.
- As the number of threads increases, multiple threads compete for memory bandwidth.
- When the number of threads exceeds the number of physical CPU cores, threads begin to share hardware resources, resulting in diminishing returns.

Performance Stats

- The task-clock value represents the total CPU time consumed by the program across all cores which is 644.11 ms.
- An average utilization of approximately 6 CPU cores indicates that multiple cores were actively involved in the computation.
- Zero context switches and CPU migrations show that threads were not frequently interrupted or moved between cores, which is desirable for performance.
- The reported page faults are minor and expected during memory allocation.

Conclusion

The OpenMP-based parallel implementation successfully reduces the execution time of the molecular dynamics force calculation compared to the serial version. While increasing the number of threads improves performance, the speedup does not scale linearly due to memory bandwidth limitations, cache contention, and hardware constraints. Performance analysis using perf stat confirms effective multi-core utilization.

Question 2: Bioinformatics - DNA Sequence Alignment (Smith-Waterman)

Problem: Implement a parallel version of the Smith-Waterman local sequence alignment algorithm for comparing two DNA sequences.

Tasks:

1. Parallelize the scoring matrix computation
2. Handle anti-dependencies in the dynamic programming approach
3. Experiment with different scheduling strategies
4. Implement wavefront parallelization (advanced)

Sol.

Smith–Waterman is a dynamic programming algorithm used in bioinformatics to find the best local alignment between two biological sequences (DNA, RNA, or protein).

In simple words:

It finds the most similar part between two DNA sequences, not necessarily aligning the full sequences.

Key points:

- Uses a 2D scoring matrix
- Each cell represents an alignment score
- Final result is the maximum value in the matrix
- Guarantees optimal local alignment

Observation

Static Scheduling:

```
Smith-Waterman Local Alignment
Sequence length : 1000
Scheduling      : OMP_SCHEDULE (runtime)

Threads Time (s)        Speedup
-------------------------------------
1       0.014064        1.00
2       0.012148        1.16
3       0.011432        1.23
4       0.011277        1.25
5       0.009166        1.53
6       0.012346        1.14
7       0.010487        1.34
8       0.010719        1.31

Alignment score (all runs): 766

 Performance counter stats for './Q2':

            436.72 msec task-clock:u                    #    4.063 CPUs utilized
                 0      context-switches:u              #    0.000 /sec
                 0      cpu-migrations:u                #    0.000 /sec
              1064      page-faults:u                   #    2.436 K/sec
```

Figure 3: Static Scheduling Performance

Under static scheduling, the workload is divided among threads at the start of execution. This results in the lowest scheduling overhead, as seen from the smallest task-clock value (436.72 ms). Speed-up increases steadily up to about 1.53 at 5 threads, after which it fluctuates. The early saturation occurs because later wavefront diagonals contain fewer cells, causing some threads to remain idle.

Dynamic Scheduling (Chunk Size = 10):

```
Smith-Waterman Local Alignment
Sequence length : 1000
Scheduling      : OMP_SCHEDULE (runtime)

Threads Time (s)        Speedup
-------------------------------------
1       0.017370        1.00
2       0.016672        1.04
3       0.015352        1.13
4       0.013566        1.28
5       0.013716        1.27
6       0.012976        1.34
7       0.014094        1.23
8       0.013248        1.31

Alignment score (all runs): 766

 Performance counter stats for './Q2':

            519.33 msec task-clock:u                    #    3.997 CPUs utilized
                 0      context-switches:u              #    0.000 /sec
                 0      cpu-migrations:u                #    0.000 /sec
              1063      page-faults:u                   #    2.047 K/sec
```

Figure 4: Dynamic Scheduling Performance

Dynamic scheduling assigns work to threads at runtime in chunks of fixed size. Although this improves load balancing, it introduces additional runtime overhead. This is reflected in a higher task-clock value (519.33 ms) compared to static scheduling. The speed-up improves slightly but remains lower overall, indicating that the workload regularity in wavefront parallelization does not justify the added scheduling overhead.

Guided Scheduling:

```
Smith-Waterman Local Alignment
Sequence length : 1000
Scheduling      : OMP_SCHEDULE (runtime)

Threads Time (s)        Speedup
-------------------------------------
1       0.014756        1.00
2       0.011729        1.26
3       0.010735        1.37
4       0.011450        1.29
5       0.011599        1.27
6       0.014132        1.04
7       0.016947        0.87
8       0.023779        0.62

Alignment score (all runs): 766

 Performance counter stats for './Q2':

            615.51 msec task-clock:u                     #    4.768 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
              1063      page-faults:u                    #    1.727 K/sec
```

Figure 5: Guided Scheduling Performance

Guided scheduling adapts chunk sizes dynamically, starting with larger chunks and reducing them over time. While useful for highly irregular workloads, it performs poorly for this problem. The task-clock value is the highest (615.51 ms), and speed-up decreases significantly at higher thread counts. This shows that scheduling overhead dominates computation, leading to reduced efficiency.

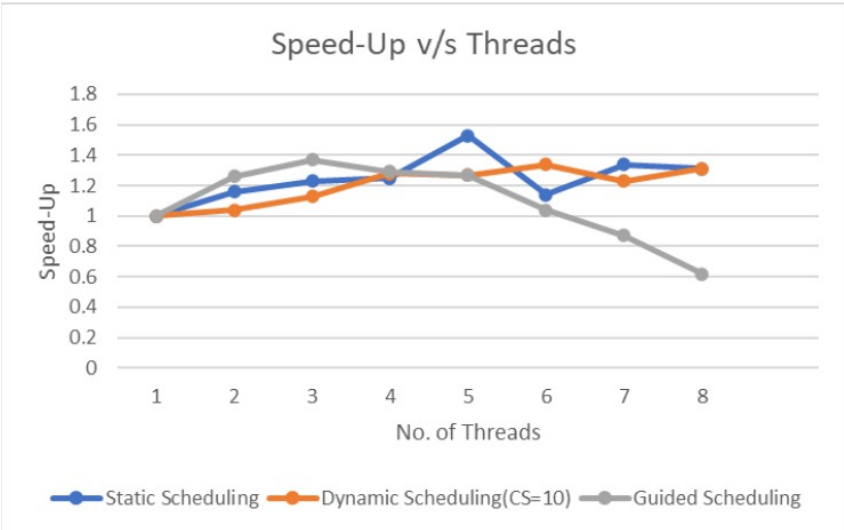

Figure 6: Speed-Up Comparison

The speed-up values obtained for the Smith–Waterman algorithm show a gradual improvement when increasing the number of threads from 1 to higher values. However, the increase is not linear and saturates early, with maximum speed-up values remaining around 1.3–1.5 depending on the scheduling policy.

This behavior occurs because Smith–Waterman is a dynamic programming algorithm with strong data dependencies. At any point in time, only the cells belonging to the same

anti-diagonal of the scoring matrix can be computed in parallel. As a result, the amount of available parallel work is limited, and additional threads beyond a certain point do not contribute effectively to performance improvement.

Performance Stats

Metric	**Explanation**
Task-Clock	Represents the total CPU time consumed by all threads. Static scheduling shows the lowest value (436.72 ms), indicating more efficient execution.
CPUs Utilized	Around 4–5 CPUs were actively used, which is expected due to diagonal-level parallelism limiting full thread utilization.
Context Switches	Zero context switches indicate stable thread execution without OS-level interruptions.
CPU Migration	Zero migrations show threads were not moved across cores, which is good for cache locality.
Page Faults	Minor page faults occurred during memory allocation and do not significantly affect performance.

Scheduling Policy Comparison

Policy	**Work Distribution**	**Runtime Overhead**	**Observed Behaviour**
Static	Assigned once at start	Lowest	Steady increase, early saturation
Dynamic (10)	Assigned at runtime in chunks	High	Slight improvement, lower than static
Guided	Adaptive chunk sizing	Medium–High	Unstable, decreases at high threads

Conclusion

The Smith–Waterman wavefront-parallel implementation benefits from parallel execution, but scalability is fundamentally limited by data dependencies inherent in the dynamic programming approach. Static scheduling performs best due to its minimal overhead and predictable workload distribution. Dynamic and guided scheduling introduce additional runtime overhead without sufficient load-balancing benefits. Performance analysis using perf stat confirms that scheduling overhead, rather than hardware limitations, is the dominant factor affecting speed-up.

Question 3: Scientific Computing - Heat Diffusion Simulation

Problem: Simulate heat diffusion in a 2D metal plate using finite difference method with parallel OpenMP implementation.

Hints for Question 3:

1. Data Dependency Analysis: The heat equation update has dependencies only on previous time step, allowing parallelization of spatial loops.
2. Reduction Operations: Use reduction(+:total) for accumulating total heat.
3. Memory Optimization: Consider loop tiling/cache blocking for better cache utilization.
4. Race Conditions: No race conditions exist as each grid point writes to unique location in next_temperature.
5. Scheduling Strategies: Experiment with schedule(static), schedule(dynamic, chunk_size), and schedule(guided).

Sol.

Heat diffusion is governed by the heat equation, which describes how heat flows from hotter regions to cooler regions over time.

The finite difference method approximates the heat equation by discretizing:

- Space → dividing the plate into a 2D grid
- Time → updating temperatures step by step

For each grid point:

- The new temperature depends on:
  - Top neighbor
  - Bottom neighbor
  - Left neighbor
  - Right neighbor
- All values are taken from the previous time step

Observation

Static Scheduling:

```
Heat Diffusion Simulation (2D)
Grid size        : 1000 x 1000
Time steps       : 500
Scheduling       : OMP_SCHEDULE (runtime)

Threads Time (s)        Speedup
--------------------------------
1       2.329841        1.00
2       1.379690        1.69
3       1.626299        1.43
4       1.399093        1.67
5       1.195315        1.95
6       1.041289        2.24
7       1.018712        2.29
8       0.919448        2.53
9       0.901785        2.58
10      0.961505        2.42

Final total heat : 100.000000

 Performance counter stats for './Q3':

          60508.09 msec task-clock:u                     #    4.731 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
              3992      page-faults:u                    #   65.975 /sec
```

Figure 7: Static Scheduling Heat Diffusion

With static scheduling, speed-up increases steadily from 1.69 at 2 threads to a maximum of approximately 2.58 at 9 threads, after which it slightly decreases. The serial execution time is 2.33 s, while the best parallel time is around 0.90 s. Static scheduling performs well because the workload is uniform across grid points and scheduling overhead is minimal.

Dynamic Scheduling (Chunk Size = 10):

```
Heat Diffusion Simulation (2D)
Grid size        : 1000 x 1000
Time steps       : 500
Scheduling       : OMP_SCHEDULE (runtime)

Threads Time (s)        Speedup
--------------------------------
1       2.305945        1.00
2       1.700432        1.36
3       1.216368        1.90
4       1.108469        2.08
5       1.101671        2.09
6       0.938906        2.46
7       0.925269        2.49
8       0.991149        2.33
9       1.016652        2.27
10      1.914574        1.20

Final total heat : 100.000000

 Performance counter stats for './Q3':

          67142.87 msec task-clock:u                     #    5.071 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
              3993      page-faults:u                    #   59.470 /sec
```

Figure 8: Dynamic Scheduling Heat Diffusion

Dynamic scheduling introduces runtime overhead due to on-demand task assignment. Although it improves load balancing, the additional scheduling cost reduces overall performance. Speed-up values are slightly lower compared to static scheduling, and execution times remain higher, indicating that the uniform nature of the heat diffusion workload does not benefit significantly from dynamic scheduling.

Guided Scheduling:

```
Heat Diffusion Simulation (2D)
Grid size        : 1000 x 1000
Time steps       : 500
Scheduling       : OMP_SCHEDULE (runtime)

Threads Time (s)        Speedup
--------------------------------
1       2.366715        1.00
2       1.474512        1.61
3       1.257713        1.88
4       1.103678        2.14
5       1.047127        2.26
6       1.035450        2.29
7       0.914819        2.59
8       0.837544        2.83
9       0.876733        2.70
10      1.047746        2.26

Final total heat : 100.000000

 Performance counter stats for './Q3':

          56339.25 msec task-clock:u                     #    4.702 CPUs utilized
                 0      context-switches:u               #    0.000 /sec
                 0      cpu-migrations:u                 #    0.000 /sec
              3993      page-faults:u                    #   70.874 /sec
```

Figure 9: Guided Scheduling Heat Diffusion

Guided scheduling shows improved speed-up at moderate thread counts, reaching approximately 2.83 at 8 threads, but performance drops at higher thread counts. This occurs because guided scheduling introduces overhead while reducing chunk sizes, which is not well-suited for the regular access pattern of the heat diffusion computation.

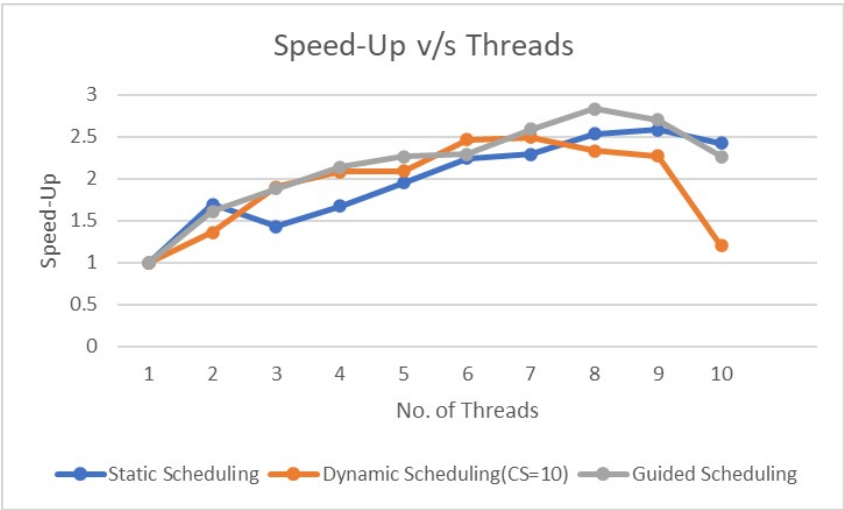

Figure 10: Speed-Up vs Threads

The output shows that execution time decreases as the number of threads increases, confirming that the heat diffusion simulation benefits from parallel execution. However, speed-up does not scale linearly and saturates after a certain number of threads. This behavior occurs because the computation is memory-bound and involves frequent access to neighboring grid points, which limits scalability due to memory bandwidth constraints.

Performance Stats

Metric	**Explanation**
Task-Clock	Values range from 60.5 ms (static) to 56.3 ms (guided), representing total CPU time consumed across all threads
CPUs Utilized	An average utilization of about 4.7 CPUs indicates partial parallel efficiency, limited by memory bandwidth and synchronization overhead
Context Switches	Zero context switches indicate stable thread execution without OS-level interruptions
CPU Migration	Zero migrations show threads were not moved across cores, which is good for cache locality
Page Faults	Minor page faults occurred during memory allocation and do not significantly affect performance

Scheduling Policy Comparison

Policy	**Work Distribution**	**Runtime Overhead**	**Observed Behaviour**
Static	Assigned once at start	Lowest	Steady improvement, early saturation
Dynamic (10)	Assigned at runtime in chunks	High	Higher overhead, lower efficiency
Guided	Adaptive chunk sizing	Medium–High	Good mid-range performance, unstable at high threads

Conclusion

The parallel OpenMP implementation of the heat diffusion simulation significantly reduces execution time compared to the serial version. Speed-up improves with increasing thread count but saturates due to memory bandwidth limitations and regular access patterns. Among the scheduling strategies tested, static scheduling provides the most stable and efficient performance for this workload, while dynamic and guided scheduling introduce additional overhead without substantial benefit. Performance analysis confirms effective multi-core utilization but highlights inherent scalability limits of memory-bound stencil computations.